Normalization

Normalization is the process of organizing a database in such a way that it reduces data redundancy, and improves data integrity, consistency, and efficiency. It involves breaking down a table into smaller tables and establishing relationships between them.

● **Data redundancy**:Redundancy means duplication of data. When there are repetitions of data in a table, then the data is said to be redundant. Aim of normalization is to reduce data redundancy.

● **Data Integrity:** Data integrity ensures that data is correct and valid, and that it is maintained in a way that meets the requirements of the Application.

● **Data consistency:** It refers to the uniformity and standardization of data across a system or organization. It ensures that the same data values are used consistently in all locations and applications where the data is used.

If there are redundant data, then problems like more memory consumption, more access time, data inconsistency and anomalies like insertion anomaly, deletion anomaly and updation anomaly might occur.

Data modification anomalies can be categorized into three types:

➢ **Insertion Anomaly**: Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.

➢ **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.

➢ **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

Consider the following table

Stu_id	Stu_name	Stu_branch	Stu_club
2018nk01	Shivani	Computer science	literature
2018nk01	Shivani	Computer science	dancing
2018nk02	Ayush	Electronics	Videography
2018nk03	Mansi	Electrical	dancing
2018nk03	Mansi	Electrical	singing
2018nk04	Gopal	Mechanical	Photography

Insertion Anomaly

If we add a new row for student Ankit who is not a part of any club, we cannot insert the row into the table as we cannot insert null in the column of stu_club. This is called insertion anomaly.

Deletion Anomaly

If we remove the photography club from the college, then we will have to delete its row from the table. But it will also delete the table of Gopal and his details. So, this is called deletion anomaly and it will make the database inconsistent.

Updation / Update Anomaly

In the above table, if Shivani changes her branch from Computer Science to Electronics, then we will have to update all the rows. If we miss any row, then Shivani will have more than one branch, which will create the update anomaly in the table.

Normal Forms:

In order to avoid all these problems, we use normalization. There are several levels of normalization, each with its own set of rules and requirements. These

levels are commonly referred to as normal forms.The most commonly used normal forms are:

- 1 NF(First Normal Form)
- 2 NF(Second Normal Form)
- 3 NF(Third Normal Form)

First Normal Form (1 NF)

If a relation contains a composite or multi-valued attribute, it violates first normal form or a relation is in first normal form if it does not contain any composite or multi-valued attribute. A relation is in first normal form if every attribute in that relation is **a single valued attribute**.

- **Example 1** – Relation STUDENT in table 1 is not in 1NF because of multi-valued attribute STUD_PHONE. Its decomposition into 1NF has been shown in table 2.

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721, 9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 1

Conversion to first normal form

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721	HARYANA	INDIA
1	RAM	9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 2

2. Second Normal Form (2NF):

A table is in 2NF if it satisfies the following conditions:

1. First Normal Form (1NF): The table must already be in 1NF, which means it contains no repeating groups within a cell and each cell has an atomic value.

2. **Functional** Dependencies: All non-key attributes (attributes not part of the primary key) must be fully functionally dependent on the entire primary key. In simpler terms, a non-key attribute should depend on all parts of the primary key, not just a portion of it.

Functional Dependency:
It is a relationship between two attributes (keys and non-keys) in a database table, where the value of one attribute determines the value of the other attribute. In other words, if we know the value of one attribute, we can determine the value of another attribute.

A table is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. Here, STUD_NAME, STUD_STATE, and STUD_COUNTRY are dependent on STUD_NO. However, STUD_PHONE is partially dependent on STUD_NO because a student can have multiple phone numbers.

To achieve 2NF, we split the table into two tables.

Student Table:

STUD_NO	STUD_NAME	STUD_STATE	STUD_COUNTRY
1	RAM	HARYANA	INDIA
2	RAM	PUNJAB	INDIA
3	SURESH	PUNJAB	INDIA

Student Phone Table:

STUD_NO	STUD_PHONE
1	9716271721
1	9871717178
2	9898297281
3	9898297281

Now both tables are in 2NF because all non-key attributes are fully dependent on the primary key of each table.

3. Third Normal Form (3NF):

- Second Normal Form (2NF): The table must already be in 2NF.
- No Transitive Dependencies: All non-key attributes must be non-transitively dependent on the primary key. This means a non-key attribute shouldn't depend on another non-key attribute, which in turn depends on the primary key.

Imagine you have a table of students in a school:

StudentID	StudentName	ClassID	ClassName	TeacherID	TeacherName
1	John	101	Math	201	Mrs. Smith
2	Alice	102	Science	202	Mr. Brown

In this table:

- `StudentID` is the primary key.
- `ClassID` is associated with `ClassName`.
- `TeacherID` is associated with `TeacherName`.

A transitive dependency exists if:

1. `StudentID` determines `ClassID` (direct dependency).
2. `ClassID` determines `TeacherID` (direct dependency).
3. Therefore, `StudentID` determines `TeacherID` indirectly through `ClassID` (transitive dependency).

In simpler terms, because `StudentID` determines `ClassID`, and `ClassID` determines `TeacherID`, `StudentID` also determines `TeacherID` through `ClassID`.

To eliminate transitive dependencies and achieve 3NF, you would break the table into smaller tables:

1. **Students Table**:

StudentID	StudentName	ClassID
1	John	101
2	Alice	102

2. **Classes Table**:

ClassID	ClassName
101	Math
102	Science

3. **Teachers Table**:

TeacherID	TeacherName
201	Mrs. Smith
202	Mr. Brown

4. **ClassTeachers Table**:

ClassID	TeacherID
101	201
102	202

EXAMPLE 2:

Initial Table (Sales Data):

Order ID	Product ID	Product Name	Customer ID	Customer Name	Salesperson ID	Salesperson Name	Order Date	Amou
1	101	Laptop	501	Alice	301	Bob	2023-01-01	1500
2	102	Mouse	502	Bob	302	Charlie	2023-01-02	20
3	101	Laptop	503	Carol	301	Bob	2023-01-03	1500
4	103	Keyboard	501	Alice	303	Dave	2023-01-04	50

First Normal Form (1NF):

In 1NF, we ensure that each column contains atomic values, and there are no repeating groups.

The initial table is already in 1NF because all attributes contain atomic values.

SECOND Normal Form (2NF):

1. SaleID → ProductID, ProductName, CustomerID, CustomerName, SalespersonID, SalespersonName, SaleDate, Amount:
    - `SaleID` is a unique identifier for each sale, so it determines all other attributes in the table.
2. ProductID → ProductName:
    - Each product has a unique `ProductID` that determines its `ProductName`.
3. CustomerID → CustomerName:
    - Each customer has a unique `CustomerID` that determines their `CustomerName`.
4. SalespersonID → SalespersonName:
    - Each salesperson has a unique `SalespersonID` that determines their `SalespersonName`.

Orders Table (2NF):

Order ID	Product ID	Customer ID	Salesperson ID	Order Date	Amount
1	101	501	301	2023-01-01	1500
2	102	502	302	2023-01-02	20
3	101	503	301	2023-01-03	1500
4	103	501	303	2023-01-04	50

Products Table:

Product ID	Product Name
101	Laptop
102	Mouse
103	Keyboard

Customers Table:

Customer ID	Customer Name
501	Alice
502	Bob
503	Carol

Salespersons Table:

Salesperson ID	Salesperson Name
301	Bob
302	Charlie
303	Dave

3 NF(no transitive dependency)

CUSTOMER, SALESPERSON AND PRODUCT TABLE ARE transitive dependency - there are only 2 attributes.
So, no transitive dependency.

IN ORDERS TABLE ANY 2 ATTRIBUTES together forms a key. So there is no scope for transitive dependency.

SO AFTER 3 NF

Orders Table (2NF):

Order ID	Product ID	Customer ID	Salesperson ID	Order Date	Amount
1	101	501	301	2023-01-01	1500
2	102	502	302	2023-01-02	20
3	101	503	301	2023-01-03	1500
4	103	501	303	2023-01-04	50

Products Table:

Product ID	Product Name
101	Laptop
102	Mouse
103	Keyboard

Customers Table:

Customer ID	Customer Name
501	Alice
502	Bob
503	Carol

Salespersons Table:

Salesperson ID	Salesperson Name
301	Bob
302	Charlie
303	Dave

LOOK AT THIS TABLE FOR 3 NF

PBrand	Ptype	Pcategory
Nike	Clothes	Tshirt
Woodland	Footwear	Shoes
Adidas	Clothes	Jacket

Table is not in 3 NF, because a transitive dependency occurs. The functional dependency in tHIS TABLE. PBrand ->Ptype, PCategory and Ptype -> Pcategory.

it is clear that table 4 is not in 3 NF. To convert the table
to 3NF, we have to decompose the table. The decomposed tables are:

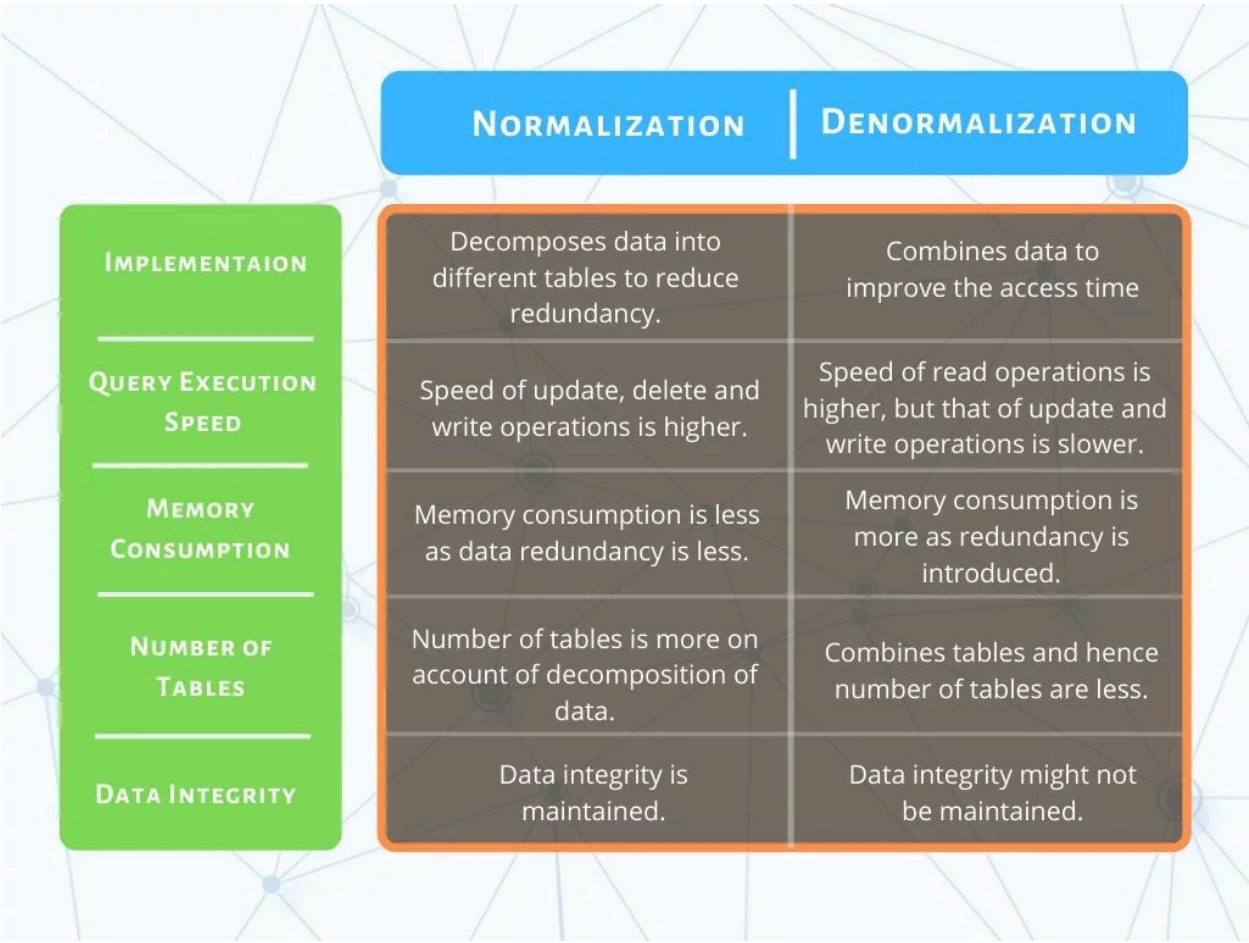
PBrand	Ptype
Nike	Clothes
Woodland	Footwear
Adidas	Clothes

Table 5

PBrand	PCategory
Nike	Tshirt
Woodland	Shoes
Adidas	Jacket

Table 6

Entri elevate



Question 1: What is database normalization and why is it important?

Answer:

Database normalization is the process of organizing the attributes and tables of a relational database to minimize redundancy and dependency. It involves dividing large tables into smaller tables and defining relationships between them according to rules designed to protect the data and make the database more flexible by eliminating redundancy and inconsistent dependency. The main goals are to reduce data redundancy, ensure data integrity, and improve query performance.

Question 2: What are the differences between First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF)?

Answer:

- First Normal Form (1NF): Ensures that the table is a flat table, meaning it contains only atomic (indivisible) values, and each column contains unique values.
- Second Normal Form (2NF): Builds on 1NF by ensuring that all non-key attributes are fully functionally dependent on the entire primary key, eliminating partial dependencies.
- Third Normal Form (3NF): Builds on 2NF by ensuring that all non-key attributes are not only fully functionally dependent on the primary key but also non-transitively dependent, meaning no transitive dependencies exist (i.e., no non-key attribute depends on another non-key attribute).

Question 3: What are the advantages and disadvantages of normalization?

Answer:
Advantages:

- Reduces data redundancy and inconsistency.
- Improves data integrity and accuracy.
- Simplifies data maintenance and updates.
- Optimizes storage space by eliminating duplicate data.

Disadvantages:

- Can lead to complex queries and joins, which might affect performance.
- May require more tables and relationships, making the database structure more complicated.
- Can result in slower read performance due to the need for multiple joins.

Question 4: What is denormalization and when might you use it?

Answer:
Denormalization is the process of deliberately introducing redundancy into a database by combining tables that have been normalized. It is used to improve read performance by reducing the number of joins needed to retrieve data. Denormalization might be used in scenarios where read performance is critical, such as in online transaction processing (OLTP) systems, or in data warehousing where large volumes of data are read frequently but updates are infrequent.

Question 5: Can you give an example of a situation where denormalization might be beneficial?

Answer:
An example where denormalization might be beneficial is in a high-traffic e-commerce website where the speed of retrieving product information, customer details, and order history is crucial. In such a scenario, combining tables like `Products`, `Customers`, and `Orders` into a single denormalized table can reduce the number of joins and significantly improve query performance, providing faster response times for users browsing and purchasing products. This trade-off is acceptable because the primary focus is on read performance rather than data redundancy or update complexity.